

LangGraph Postgres Runtime

架构完整分析报告

Python → gRPC → Go 双进程协作全链路详解

2026-06-15

目录

1. 整体架构概览
2. 进程启动与生命周期
3. gRPC 通信机制详解
4. Python → Go: 元数据操作
5. Python 端: 图执行全流程
6. Go → Python: 反向调用
7. Checkpoint 存储的三条路径
8. 流事件分发全链路
9. 认证与加密
10. 环境变量与配置
11. 与自研 postgres_py 的对比
12. 总结

1. 整体架构概览

LangGraph Postgres Runtime 采用 Python + Go 双进程架构，两个进程通过 gRPC 通信，共享同一个 PostgreSQL 数据库。

1.1 架构图



1.2 两个进程的职责

进程	语言	端口	职责	开源
Python	Python	HTTP:8000 GRPC:50071	API接口、图执行、Checkpoint	□ 开源

进程	语言	端口	职责	开源
Go	Go	GRPC:50051	元数据CRUD、事件分发、TTL	□ 闭源

核心原则：图执行在 Python，数据管理在 Go，两者共享 PostgreSQL。

2. 进程启动与生命周期

2.1 启动顺序

1. Go 进程先启动 (core-api-grpc)
 - 监听 :50051
 - 连接 PostgreSQL
 - 运行数据库迁移
2. Python 进程启动
 - 连接 Go gRPC 服务 (:50051)
 - 健康检查 (最多等60秒)
 - 启动 Python gRPC 服务 (:50071, 可选)
 - 注册 Checkpointer/Encryption Servicer
 - 加载用户图
 - 启动 Worker 消费队列

2.2 Python 进程启动细节 (lifespan)

```
async def lifespan(app):
# 1. 启动 HTTP 客户端
await start_http_client()

# 2. 等待 Go gRPC 服务就绪
await wait_until_grpc_ready(timeout=60)

# 3. 启动 Python gRPC 服务 (可选)
if PYTHON_GRPC_SERVER_ENABLED: # 加密或自定义checkpointer时启用
await start_python_grpc_server(port=50071)

# 4. 加载用户图
await collect_graphs_from_env()

# 5. 启动后台任务
async with SimpleTaskGroup(cancel=True) as tg:
tg.create_task(queue()) # Worker 队列消费
tg.create_task(cron_scheduler()) # 定时任务
yield # 应用运行中

# 6. 关闭
await stop_python_grpc_server()
await stop_http_client()
```

2.3 Python gRPC 服务何时启动？

Python gRPC 服务 (:50071) 不是总是启动，仅在以下条件满足时启动：

```
PYTHON_GRPC_SERVER_ENABLED = bool(
LANGGRAPH_ENCRYPTION or # 配置了自定义加密
USE_CUSTOM_CHECKPOINTER # 配置了自定义 checkpointer
)
```

默认 Postgres 后端（无自定义加密/checkpointer）时，Python gRPC 服务不启动，因为 Go 不需要反向调用 Python。

3. gRPC 通信机制详解

3.1 双向 gRPC 通信

两个进程之间有 两条 gRPC 通道：

方向	Python 端	Go 端	用途
Python → Go	gRPC Client (:50051)	gRPC Server	元数据CRUD、任务队列、事件发布
Go → Python	gRPC Server (:50071)	gRPC Client	Checkpoint读写、加解密

3.2 Python → Go 的 7 个 gRPC Stub

```
class GrpcClient:
    self._assistants_stub = AssistantsStub(channel) # 助手管理
    self._runs_stub = RunsStub(channel) # Run 管理
    self._threads_stub = ThreadsStub(channel) # 线程管理
    self._crons_stub = CronsStub(channel) # 定时任务
    self._admin_stub = AdminStub(channel) # 运维管理
    self._cache_stub = CacheStub(channel) # 缓存
    self._checkpointer_stub = CheckpointerStub(channel) # Checkpoint(仅Mongo)
```

3.3 客户端连接池

Python 维护一个 gRPC 客户端池，轮询分发请求：

```
class GrpcClientPool:
    pool_size = 5 # 默认5个客户端实例

    async def get_client(self) -> GrpcClient:
        idx = self._current_index % self.pool_size # 轮询
        self._current_index = idx + 1
        return self.clients[idx]
```

3.4 Go → Python 的 2 个 Servicer

Python gRPC 服务注册的 Servicer（条件注册）：

Servicer	注册条件	提供的方法
CheckpointerServicer	USE_CUSTOM_CHECKPOINTER	Put, PutWrites, List, GetTuple, DeleteThread, CopyThread, Prune
EncryptionServicer	LANGGRAPH_ENCRYPTION	EncryptJSON, DecryptJSON

4. Python → Go: 元数据操作

4.1 调用模式

所有元数据操作（Thread/Run/Assistant/Cron 的 CRUD）都通过 gRPC 调用 Go 服务：

```
# 典型调用流程
async def Threads.get(conn, thread_id, ctx=None):
# 1. 构造 protobuf 请求
request = pb.GetThreadRequest(
    thread_id=pb.UUID(value=str(thread_id)),
    filters=auth_filters,
)
# 2. 获取 gRPC 客户端
client = await get_shared_client()
# 3. 调用 Go 服务
response = await client.threads.Get(request)
# 4. 转换响应
yield proto_to_thread(response.thread)
```

4.2 各操作的 gRPC 映射

Python ops 方法	gRPC 调用	Go 执行
Assistants.put()	client.assistants.Create()	INSERT INTO assistants
Assistants.get()	client.assistants.Get()	SELECT FROM assistants
Assistants.search()	client.assistants.Search()	SELECT + 分页
Threads.put()	client.threads.Create()	INSERT INTO threads
Threads.get()	client.threads.Get()	SELECT FROM threads
Threads.delete()	client.threads.Delete()	DELETE + 级联清理
Runs.put()	client.runs.Create()	INSERT INTO runs + 入队
Runs.next()	client.runs.Next()	SELECT pending + 更新状态
Runs.Stream.publish()	client.runs.Publish()	Redis XADD + 分发
Crons.put()	client.crons.Create()	INSERT INTO crons

4.3 Runs.put() 的完整流程

创建 Run 是最复杂的操作，涉及认证、加密、入队：

```
async def Runs.put(conn, assistant_id, kwargs, *, thread_id, ...):
# 1. Thread 认证
auth_filters = await Runs.handle_event(ctx, 'create_run', value)

# 2. Assistant 认证 (自动检查助手归属)
assistant_auth_filters = await auth_handle_event(
    AuthContext(resource='assistants', action='read'), ...
)

# 3. 构建加密上下文
enc_ctx = build_encryption_context('run')

# 4. gRPC 调用 Go
```

```
response = await client.runs.Create(pb.CreateRunRequest(
    assistant_id=..., kwargs_json=...,
    thread_filters=auth_filters,
    assistant_filters=assistant_auth_filters,
    encryption_context=enc_ctx,
))
```

```
# 5. 返回创建的 Run
for run in response.runs:
    yield proto_to_run(run)
```


5. Python 端：图执行全流程

图在 Python 进程的 Worker 中执行，Go 完全不参与图执行。

5.1 Worker 获取任务

Worker 通过 `Runs.next()` 从 Go 服务获取待执行的 Run：

```
# gRPC 实现
async def Runs.next(wait, limit):
    client = await get_shared_client()
    response = await client.runs.Next(
        pb.NextRunRequest(wait=wait, limit=limit)
    )
    for run_with_attempt in response.runs:
        yield proto_to_run(run_with_attempt.run), run_with_attempt.attempt
```

5.2 进入 Run 执行上下文

`Runs.enter()` 打开一个双向 gRPC 流，用于接收中断/回滚信号：

```
@asynccontextmanager
async def Runs.enter(run_id, thread_id, loop, resumable):
    done = ValueEvent() # 中断信号

    # 打开双向流到 Go 服务
    client = await get_shared_client()
    enter_stream = client.runs.Enter(
        pb.EnterRunRequest(run_id=..., thread_id=..., resumable=...)
    )

    # 后台监听控制信号
    async def listen_for_signals():
        async for event in enter_stream:
            if event.action == 'interrupt':
                done.set(UserInterrupt()) # 中断!
            elif event.action == 'rollback':
                done.set(UserRollback()) # 回滚!

    listener = asyncio.create_task(listen_for_signals())
    yield done # 返回给 Worker
    await Runs.mark_done(run_id, thread_id, resumable)
```

5.3 图执行核心 (worker.py + stream.py)

```
async def worker(run, attempt, main_loop):
    # 1. 解密 run kwargs
    run['kwargs'] = await decrypt_response(run['kwargs'], ...)

    # 2. 进入 Run 上下文 (接收中断信号)
    async with Runs.enter(run_id, thread_id, loop, resumable) as done:

        # 3. 创建图执行流
        stream = astream_state(run, attempt, done)

        # 4. 消费流 (发布事件)
        await consume(stream, run_id, resumable, stream_modes, thread_id)

    # 5. 更新状态
    async with connect() as conn:
```

```
await Threads.set_joint_status(conn, thread_id, run_id, 'success', ...)
```

5.4 astream_state() - 图的实际执行

```
async def astream_state(run, attempt, done, ...):
    # 1. 获取图 (Python 对象)
    graph = await get_graph(graph_id, config,
                             checkpoint=await get_checkpoint(), # Python Checkpointer!
    )

    # 2. 执行图 (纯 Python! )
    async for event in graph.astream(input, config, stream_mode=...):
        # 3. 检查中断
        event = await wait_if_not_done(anext(stream), done)
        # 如果 done 被设置 (中断/回滚), 立即停止

        # 4. 处理 checkpoint
        if mode == 'debug' and chunk['type'] == 'checkpoint':
            on_checkpoint(chunk) # 保存到 run 结果

        # 5. yield 事件给 consume()
        yield mode, chunk
```

5.5 consume() - 事件发布

```
async def consume(stream, run_id, resumable, stream_modes, thread_id):
    async for mode, payload in stream:
        payload_bytes = json_dumps(payload)
        # 通过 gRPC 发布到 Go 服务
        await Runs.Stream.publish(
            run_id, mode, payload_bytes,
            thread_id=thread_id, resumable=is_resumable,
        )
```

5.6 完整时序图

```
Worker Runs.next() Go:Next() Go:SQL PG
| | | | |
| 获取任务 | | | |
|-----□| gRPC Next | | |
| |-----□| SELECT pending | |
| | |-----□| |
| | |□-----| |
| (run,attempt) | □-----| | |
| □-----| | | |
| | | |
| Runs.enter() | | | |
|-----□| 标记running | |
| | |-----□| |
| done=ValueEvent | | |
| | | |
| graph.astream() [Python执行图] | | |
|-----□|
| checkpointer.aput() [Python直写PG] | |
|-----□|
| | | |
| Runs.Stream.publish(event) | | |
|-----□| 分发给订阅者 | |
| | | |
| set_joint_status('success') | | |
```

_____ □	UPDATE status		
		_____ □	
done			

6. Go → Python: 反向调用

Go 服务在某些场景下需要反向调用 Python，通过 Python gRPC 服务 (:50071) 实现。

6.1 何时发生反向调用？

场景	Go 调用 Python	条件
读取 Checkpoint	CheckpointerServicer.GetTuple()	使用自定义 Checkpointer (如 Redis)
写入 Checkpoint	CheckpointerServicer.Put()	同上
删除线程 Checkpoint	CheckpointerServicer.DeleteThread())	同上
加密数据	EncryptionServicer.EncryptJSON()	配置了自定义加密
解密数据	EncryptionServicer.DecryptJSON()	同上

6.2 默认 Postgres 后端：无需反向调用

默认 Postgres 后端时，Go 和 Python 各自直连 PostgreSQL，Go 不需要反向调用 Python！

原因：

- Python Worker 直接用 AsyncPostgresSaver 读写 checkpoint
- Go 服务直接用 pgx 读写元数据表
- 两者共享同一个 PostgreSQL 数据库，通过数据库做数据交接
- Python gRPC 服务 (:50071) 不启动

6.3 自定义 Checkpointer 时：Go 需要反向调用



7. Checkpoint 存储的三条路径

Checkpoint 的存储方式取决于配置，有三种路径：

7.1 路径 A：默认 Postgres（最常见）

配置: LANGGRAPH_RUNTIME_EDITION=postgres (无自定义checkpointer)

流程:

Python Worker

|

| checkpointer.aput() / aget_tuple()

| (AsyncPostgresSaver 直连 PG)

▼

PostgreSQL

|— checkpoints 表

|— checkpoint_blobs 表

|— checkpoint_writes 表

Go 服务: 不参与 checkpoint 操作

Python gRPC 服务: 不启动

7.2 路径 B：自定义 Checkpointer（如 Redis）

配置: LANGGRAPH_CHECKPOINTER={"backend":"custom","path":"my_checkpointer"}

流程:

Python Worker

|

| checkpointer.aput() (自定义实现)

▼

Redis / S3 / 其他存储

Go 服务需要 checkpoint 时:

Go —gRPC— Python (:50071) — checkpointer.aget_tuple()

Python gRPC 服务: 启动 (注册 CheckpointerServicer)

7.3 路径 C：Mongo 后端

配置: LANGGRAPH_CHECKPOINTER={"backend":"mongo"}

流程:

Python Worker

|

| GrpcCheckpointner (gRPC 客户端)

▼

Go 服务 (:50051)

|

▼

MongoDB

注意: 这是唯一 Python 通过 gRPC 调 Go 做 checkpoint 的路径

7.4 三条路径对比

路径	Checkpoint 存储	Python 写入	Go 读取	Python gRPC Server
A: 默认PG	PostgreSQL	直连 psycopg3	直连 pgx	不启动
B: 自定义	Redis/S3/...	直连自定义实现	gRPC→Python	启动
C: Mongo	MongoDB	gRPC→Go→Mongo	直连 Mongo	不启动(注)

8. 流事件分发全链路

图执行产生的事件需要实时推送给客户端（SDK / LangGraph Studio）。

8.1 事件产生到客户端的完整路径

1. 图执行产生事件 (Python)
graph.astream() → yield (mode, payload)
2. consume() 序列化并发布 (Python)
payload_bytes = json_dumpb(payload)
await Runs.Stream.publish(run_id, mode, payload_bytes, ...)
3. gRPC 发送到 Go 服务 (Python → Go)
client = await get_shared_client()
await client.runs.Publish(PublishStreamEventRequest(...))
4. Go 服务分发事件 (Go)
- 存储到 Redis Stream (可恢复, TTL 120秒)
- 推送给 SSE/WebSocket 订阅者
5. 客户端接收 (SDK / Studio)
GET /threads/{id}/runs/{id}/stream (SSE)

8.2 事件类型

事件类型	说明	产生时机
values	图的完整状态	每个 superstep 后
updates	增量更新	每个节点执行后
messages	消息流 (流式输出)	LLM 生成 token 时
debug	调试信息 (含 checkpoint)	每步执行后
interrupt	中断事件	遇到 interrupt() 时
error	错误事件	执行异常时
metadata	运行元数据	Run 开始时

8.3 中断处理机制

当用户请求中断 Run 时：

1. 用户调用 POST /runs/cancel (action='interrupt')
2. Python API → gRPC → Go 服务
client.runs.Cancel(CancelRunRequest(action=INTERRUPT))
3. Go 服务通过 Enter 流发送中断信号
(Go 已持有 Runs.enter() 的双向流)
4. Python Worker 收到信号
listen_for_signals() → done.set(UserInterrupt())
5. 图执行停止
wait_if_not_done() 检测到 done 被设置
→ 抛出 UserInterrupt 异常

→ 图在当前 checkpoint 处暂停

9. 认证与加密

9.1 认证流程

每个 ops 方法在执行前都会调用 handle_event() 进行认证:

```
# Python 端
auth_filters = await Threads.handle_event(ctx, 'read', value)
# 返回过滤条件, 如 {"owner": "user123"}

# 转换为 protobuf 格式
proto_filters = _filters_to_proto(auth_filters)

# 发送给 Go 服务
response = await client.threads.Get(
    GetThreadRequest(thread_id=..., filters=proto_filters)
)
# Go 服务在 SQL 查询中应用过滤条件
```

9.2 加密机制

加密有两种模式:

模式	配置	处理位置	Python gRPC Server
AES 加密	LANGGRAPH_AES_KEY	Python 端 (中间件)	不启动
自定义加密	LANGGRAPH_ENCRYPTION	Go→Python (gRPC)	启动 EncryptionServicer

10. 环境变量与配置

环境变量	默认值	说明
LANGGRAPH_RUNTIME_EDITION	inmem	运行时版本: inmem/postgres
LSD_GRPC_SERVER_ADDRESS	localhost:50051	Go gRPC 服务地址
GRPC_CLIENT_POOL_SIZE	5	gRPC 客户端池大小
PYTHON_GRPC_SERVER_PORT	50071	Python gRPC 服务端口
PYTHON_GRPC_BIND_HOST	127.0.0.1	Python gRPC 绑定地址
DATABASE_URI	-	PostgreSQL 连接串
N_JOBS_PER_WORKER	10	Worker 并发数
LANGGRAPH_CHECKPOINTER	-	自定义 Checkpointer 配置
LANGGRAPH_ENCRYPTION	-	自定义加密配置
LANGGRAPH_AES_KEY	-	AES 加密密钥

11. 与自研 postgres_py 的对比

11.1 架构对比

维度	官方 Postgres Runtime	自研 postgres_py
进程数	2 (Python + Go)	1 (Python)
通信方式	gRPC 双向	无 (函数调用)
元数据存储	Go → PostgreSQL	Python → PostgreSQL
Checkpoint	Python → PostgreSQL (直连)	Python → PostgreSQL (直连)
事件分发	Python → gRPC → Go → SSE	Python → Redis Pub/Sub
任务队列	Go 内部队列	Redis Streams
TTL 清理	Go 后台任务	待实现
部署	Docker (必须)	pip install (可选 Docker)
调试	跨语言, 复杂	纯 Python, 简单

11.2 数据流对比

```
# 官方: 创建 Thread
Python API → gRPC → Go → SQL → PostgreSQL

# 自研: 创建 Thread
Python API → ops.Threads.put() → SQL → PostgreSQL

# 官方: 执行图
Worker → graph.astream() → Checkpointer → PostgreSQL (直连)
→ Runs.Stream.publish() → gRPC → Go → SSE

# 自研: 执行图
Worker → graph.astream() → Checkpointer → PostgreSQL (直连)
→ Runs.Stream.publish() → Redis Pub/Sub → SSE
```

12. 总结

12.1 核心架构原则

- 图执行在 Python: 用户代码、LangChain、LLM 调用都在 Python
- 数据管理在 Go: 元数据 CRUD、任务队列、事件分发由 Go 处理
- 共享 PostgreSQL: 两个进程直连同一个数据库, 通过数据做交接
- gRPC 是胶水: Python 通过 gRPC 调 Go 做元数据操作, Go 偶尔反向调 Python
- 默认无需反向调用: 标准 Postgres 后端时, Go 不需要调 Python

12.2 为什么要这样设计?

- 性能: Go 处理大量并发连接和数据库操作更高效
- 隔离: 图执行崩溃不影响数据管理服务

- 可扩展：可以独立扩展 Go 服务实例数
- 商业保护：核心持久化逻辑闭源，Python 层开源

— 报告结束 —